

Process Scheduling

Module 4

Operating Systems

Chapter Outline

- Scheduler
 - Scheduling algorithm
 - Objectives of Scheduling
 - Criteria for scheduling
 - CPU-bound vs I/O-bound processes
- Types of Scheduling
- Process scheduling queues
 - FCFS
 - SJF
 - Priority
 - Round Robin
 - Multilevel feedback queues scheduling
- BSD Unix scheduling
- Multiple processor scheduling

Scheduling

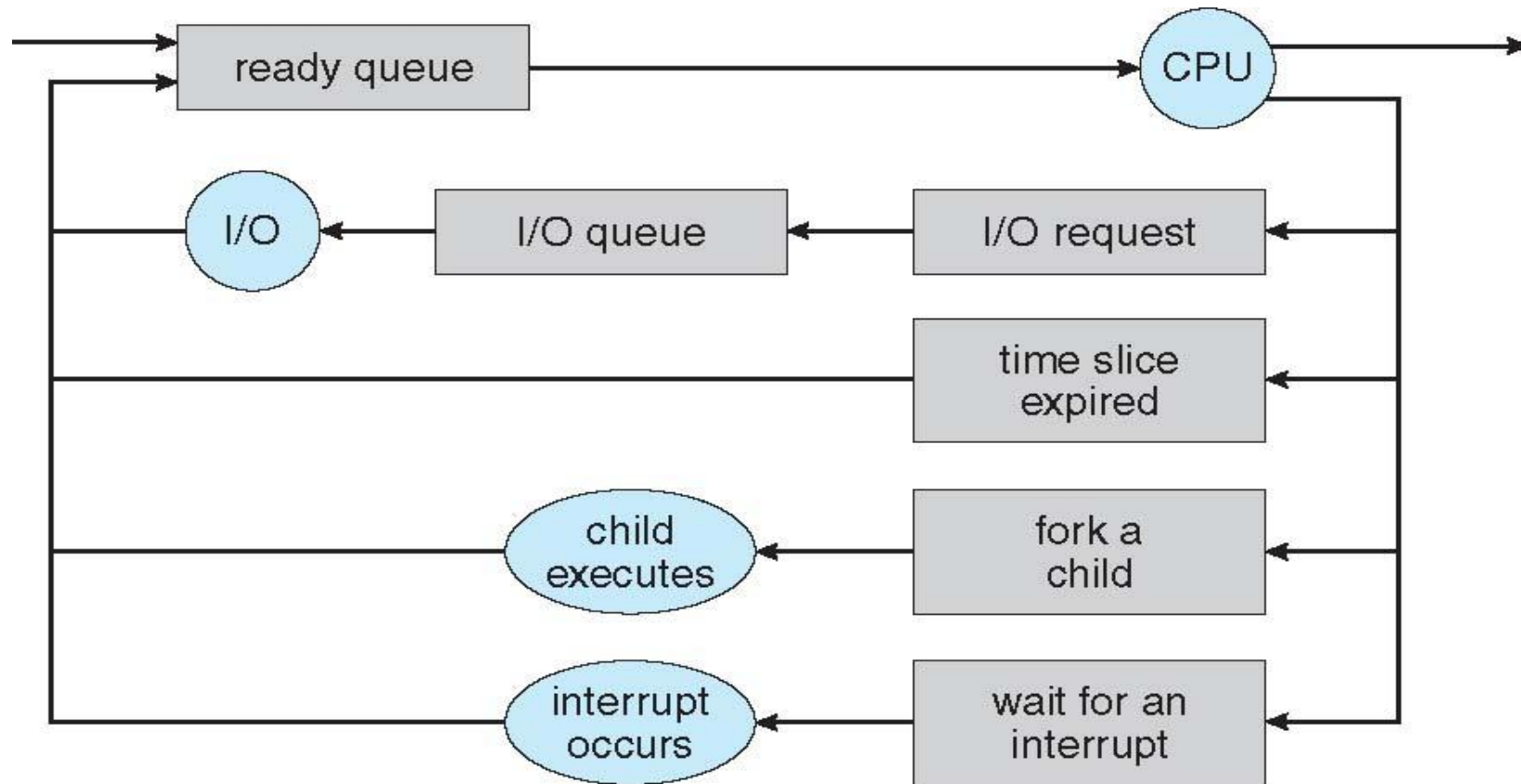
- Scheduler

- The scheduler is part of the operating system, concerned with which process to run next if multiple run-able processes are in ready queue
- Scheduler is concerned with deciding on policy, not providing a mechanism
- It is important because it can have a big effect on resource utilization and overall performance of the system

- Scheduling Algorithm

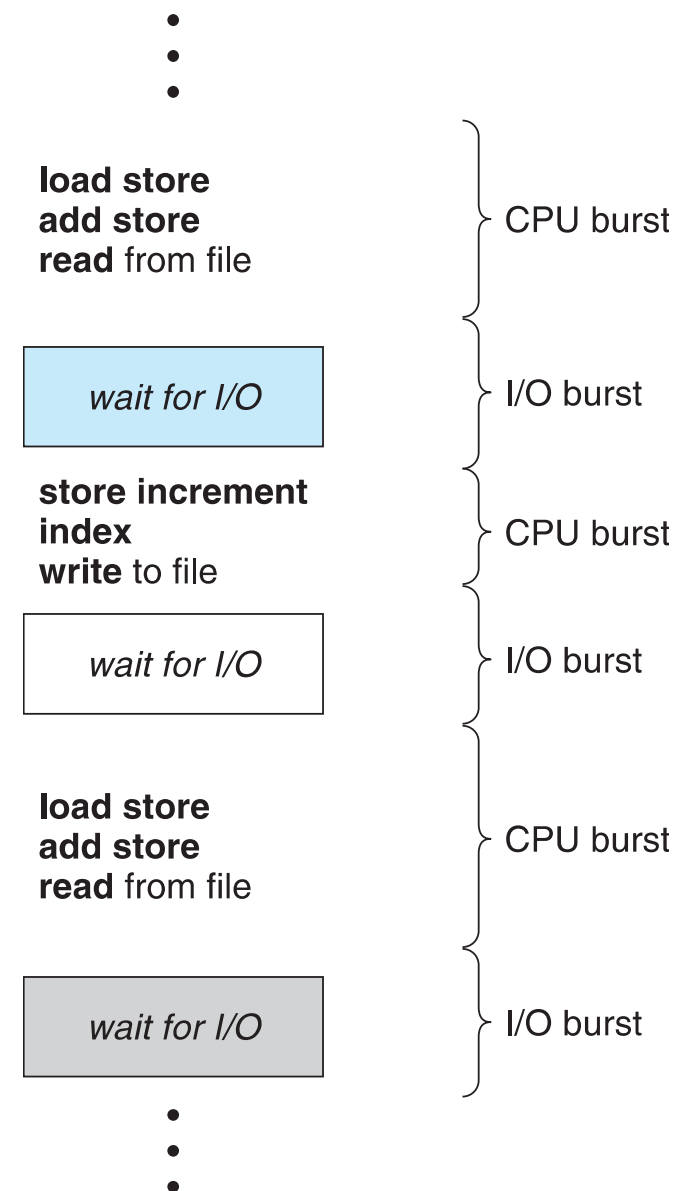
- In a multiprogramming computer several processes will compete for use of the processor
- At any time only one process will be running while other will wait for process to be free
- Scheduling algorithm is logic that determines the order in which processes should run when multiple processes are there
- The aim of process scheduling is to achieve objectives such as maximum CPU utilization and to improve its performance

Process Scheduling

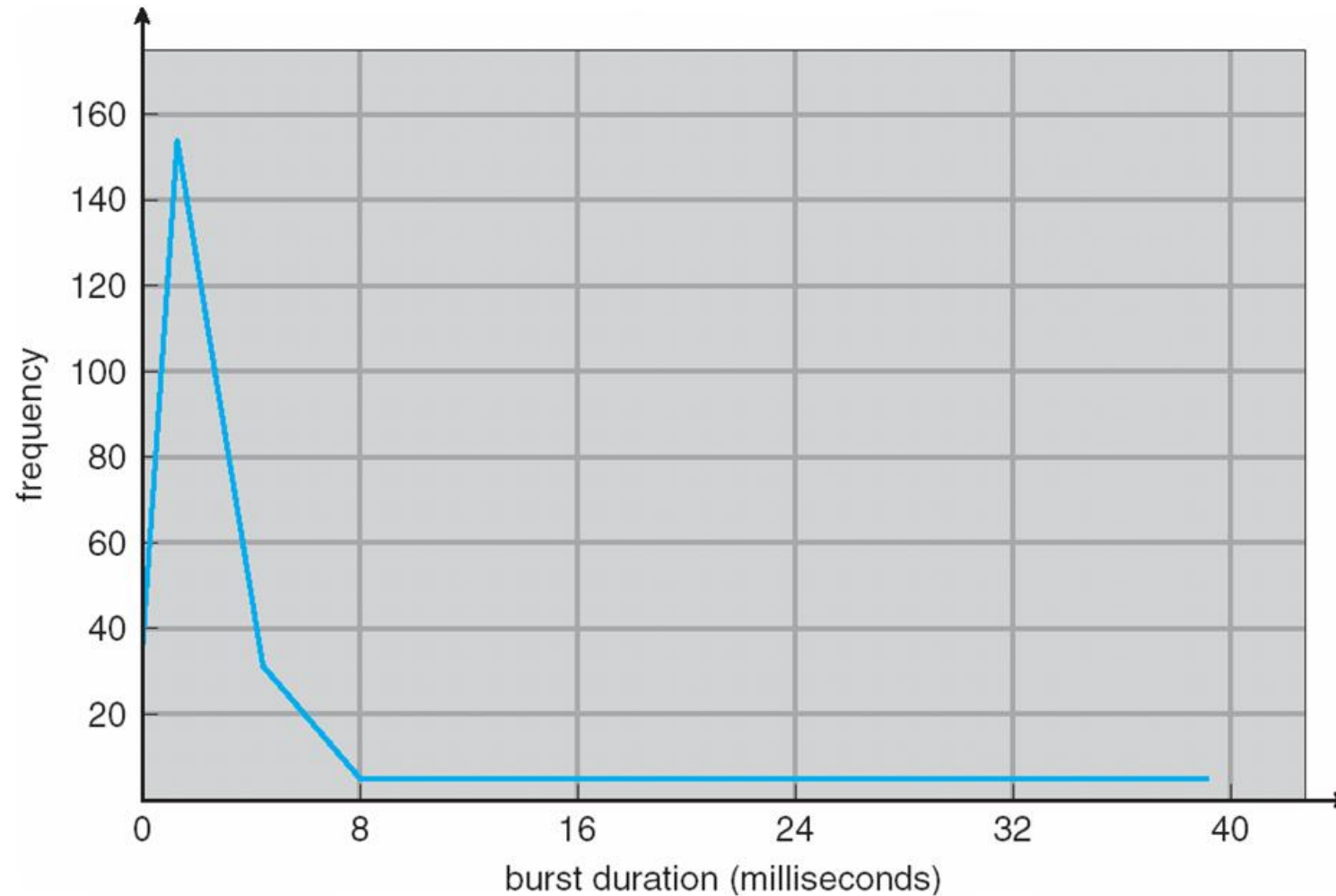


Process Scheduling

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a cycle of CPU execution and I/O wait
- CPU burst followed by I/O burst
- CPU burst distribution is of main concern



Histogram of CPU-burst Times



CPU Scheduler

- Short-term scheduler selects from among the processes in ready queue, and allocates the CPU to one of them
 - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
 - Switches from running to waiting state
 - Switches from running to ready state
 - Switches from waiting to ready
 - Terminates
- Scheduling under 1 and 4 is non-preemptive
- All other scheduling is preemptive
 - Consider access to shared data
 - Consider preemption while in kernel mode
 - Consider interrupts occurring during crucial OS activities

Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - switching context
 - switching to user mode
 - jumping to the proper location in the user program to restart that program
- Dispatch latency
 - time it takes for the dispatcher to stop one process and start another running

Scheduling Performance

- CPU utilization
 - keep the CPU as busy as possible
- Throughput
 - # of processes that complete their execution per time unit
- Turnaround time
 - amount of time to execute a particular process
- Waiting time
 - amount of time a process has been waiting in the ready queue
- Response time
 - amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

Scheduling Algorithm Objectives

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Criteria for Scheduling

- Priority
 - Priority assigned to that process or job
- Class
 - Class of job, i.e. batch or on-line or real-time
- Resource requirements
 - E.g. expected run time, memory required, etc
- I/O or CPU bound
 - i.e. whether job is I/O bound or CPU bound
- Resources used to date
 - i.e. the amount of processor time already consumed
- Waiting time to date
 - i.e. the amount of time spent waiting for service so far

Scheduling Philosophies

- Preemptive
 - The system may preempt the CPU from a process at any time;
 - Prevents any one process from using the CPU for too long
 - Can lead to race condition (require synchronization techniques)
- Non-preemptive
 - Each process voluntarily gives up the CPU
 - It is;
 - Simple
 - Easy to implement
 - Not suitable for multi-user systems
 - If the running process become blocked next process can be scheduled

Process Scheduling Queues

- Job queue
 - Set of all the processes in the system
 - It also includes the processes from the queues given below
- Ready queue
 - Set of all the processes residing in main memory (ready or waiting)
- Device queue
 - Set of processes waiting for an I/O devices

Levels of Scheduling

- Scheduling can be exercised at three levels
- High-level scheduling (Long-term scheduler)
 - Allow limited number of processes in the ready queue to compete
 - It disallows processes beyond a certain limit for batch processes first and in the end also the interactive processes
 - Invoke less frequently
 - May be slow
- Medium-level scheduling (Medium-term scheduler)
 - It is concerned with decision to temporarily remove a process from the system (to reduce system load)
 - To reintroduce a process or swapping in
 - What if there is room in memory and both a new process as well as swapped out process want to be loaded
 - In this case, medium level scheduler has to work in close conjunction with high level scheduler

Levels of Scheduling

- Low level-scheduling (Short-term scheduler)
 - Handles the decisions of which process is to be assigned to the processor (dispatcher)
 - Invoke very frequently
 - Must execute fast
 - Its scheduling could be preemptive or non-preemptive
 - Dispatch latency should be minimum
 - Dispatch latency is the time it takes for the dispatcher to stop one process and start another running
 - Example:
 - Assume a process runs for 90ms and scheduler run for 10ms
 - Overheads = $10/(90+10) = 10\%$
- Any OS may use one or all of these levels, depending upon the desired
- These levels have to interact amongst themselves quite closely to ensure that the computing resources are managed optimally
- The exact algorithms for these and interaction between them are quite complex

Schedulers in Operating System



Scheduler	Key Decision	Runs On	Frequency
Long-Term	Which jobs to admit	Disk → Main Memory	Low
Medium-Term	Which jobs to suspend/resume	Main Memory ↔ Disk	Medium
Short-Term	Which process gets CPU	Ready Queue → CPU	High

Scheduling Algorithms

First Come First Serve Scheduling

- The FCFS simply assign the processor to the process which is first in ready queue
- When the processor is free, the next process at the head of the ready queue will be selected

- Consider the following set of three processes that arrive at time '0'

Process ID	Priority	Burst Time
P1	2	30 ms
P2	3	6 ms
P3	1	4 ms



- The waiting time for each process is P1=0, P2=30, P3=36
- The average waiting time is $(0+30+36)/3 = 22\text{ms}$
- The average waiting time of FCFS may vary substantially if the process's CPU burst time vary greatly

First Come First Serve Scheduling

- Salient points of FCFS
 - It is the simplest scheduling algorithm
 - Also known as the first in first out (FIFO)
 - FCFS is non-preemptive
 - It performs much better for long processes than short processes
 - It tends to favor CPU-bound processes over I/O bound
 - It is not suitable for time sharing system
 - FCFS is rarely used on its own but is effective if combined with other scheme

Practice Example (FCFS)

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	5
P2	2	3
P3	4	8

1. Sort Processes by Arrival Time
2. Design Gantt Chart
3. Calculate Waiting Time & Turnaround Time
4. Calculate Averages (Average waiting & Turnaround Time)

Solution

Step 1: Sort Processes by Arrival Time

- Since FCFS executes the process that **arrives first**, we arrange them by **arrival order**:

P1 → P2 → P3

Step 2: Gantt Chart

Let's schedule them in order:

P1	P2	P3
0 5	8	16

Step 3: Calculate Waiting Time & Turnaround Time

Process	Arrival Time	Burst Time	Start Time	Finish Time	Waiting Time	Turnaround Time
P1	0	5	0	5	0	5
P2	2	3	5	8	$5 - 2 = 3$	$8 - 2 = 6$
P3	4	8	8	16	$8 - 4 = 4$	$16 - 4 = 12$

Step 4: Calculate Averages

- **Average Waiting Time** = $(0 + 3 + 4) / 3 = 2.33$ ms
- **Average Turnaround Time** = $(5 + 6 + 12) / 3 = 7.67$ ms

Shortest Job First Scheduling

- This algorithm associates with shortest job first
- When the CPU is available, it is assigned to the process that has smallest CPU burst time
- If two processes have same CPU-burst then FCFS scheduling is used along with this to break the tie
- Moving a short process before a long one
 - Decrease the waiting time of short process
 - Increase the waiting time of long process
 - Consequently, the average time decreases
- This will result in a smaller number of PCB's in the ready or blocked queues
- The search time will be smaller, thus improving the response time
- The number of satisfied user will increase
- A long job in the queue may delayed indefinitely by a succession of smaller jobs arriving in ready queue
- This can be avoided by setting higher external priority to those important jobs
- The OS at any time can calculates a resultant priority based on both
- Also known as shortest job next

Shortest Job First Scheduling

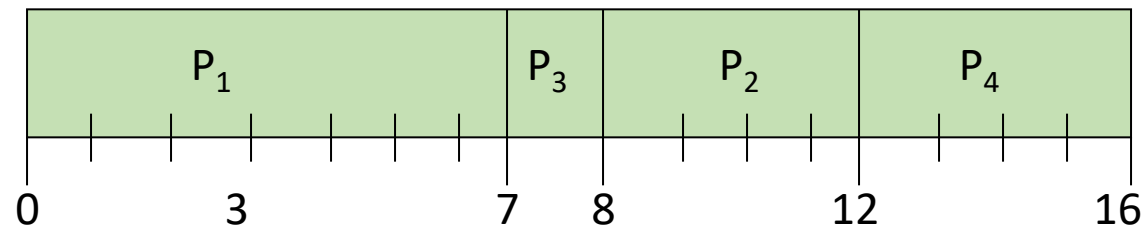
- Salient Points of SJF
 - SJF may be preemptive or non-preemptive
 - It will preempt the currently executing process if its remaining time is greater than that of a newly arrived process
 - This scheme is known as shortest remaining time first
 - SJF is optimal in sense that it yields the smallest average waiting time
 - In general, it is difficult to predict the CPU time requirement for a process
 - SJF is optimal for batch jobs for which the run times are known in advance
 - SJF reduces average waiting time over FCFS

Shortest Job First Scheduling

Non-Preemptive SJF Scheduling

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)



- Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

Completion, Turnaround and Waiting times

Turnaround = Completion – Arrival

Waiting = Turnaround – Burst

Process	Arrival	Burst	Completion C	Turnaround (C-A)	Waiting (TAT – Burst)
P ₁	0	7	7	7 – 0 = 7	7 – 7 = 0
P ₂	2	4	12	12 – 2 = 10	10 – 4 = 6
P ₃	4	1	8	8 – 4 = 4	4 – 1 = 3
P ₄	5	4	16	16 – 5 = 11	11 – 4 = 7

Totals / averages:

- Average **turnaround time** = $(7 + 10 + 4 + 11) / 4 = \mathbf{8}$ units
- Average **waiting time** = $(0 + 6 + 3 + 7) / 4 = \mathbf{4}$ units

Final answer

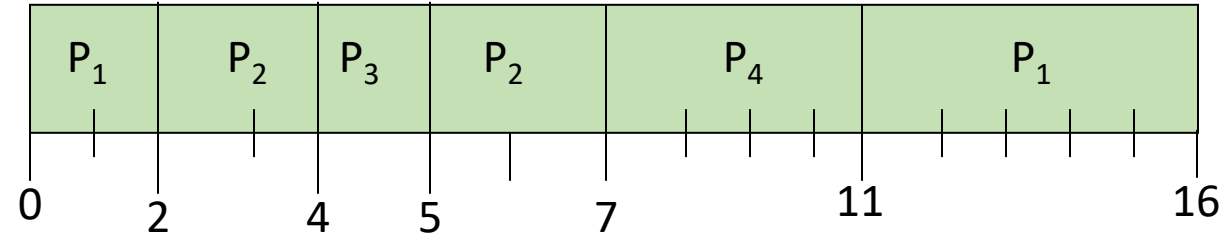
- Average waiting time = **4 units**
- Average turnaround time = **8 units**

Shortest Job First Scheduling

Preemptive SJF Scheduling

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



Completion, Turnaround and Waiting times

Compute completion time (C), turnaround = C – arrival, waiting = turnaround – burst.

Process	Arrival	Burst	Completion C	Turnaround (C-A)	Waiting (TAT-Burst)
P ₁	0	7	16	16	16 – 7 = 9
P ₂	2	4	7	7 – 2 = 5	5 – 4 = 1
P ₃	4	1	5	5 – 4 = 1	1 – 1 = 0
P ₄	5	4	11	11 – 5 = 6	6 – 4 = 2

Totals / averages:

- Average **turnaround time** = $(16 + 5 + 1 + 6) / 4 = 28 / 4 = 7$
- Average **waiting time** = $(9 + 1 + 0 + 2) / 4 = 12 / 4 = 3$

Average waiting time = **3** units. Average turnaround time = **7** units.

Round Robin Scheduling

- The Round-Robin (RR) scheduling is designed especially for time sharing systems
- In this algorithm there is small unit of time known as time quantum
- A time quantum is generally up to 10-100ms
- CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 quantum

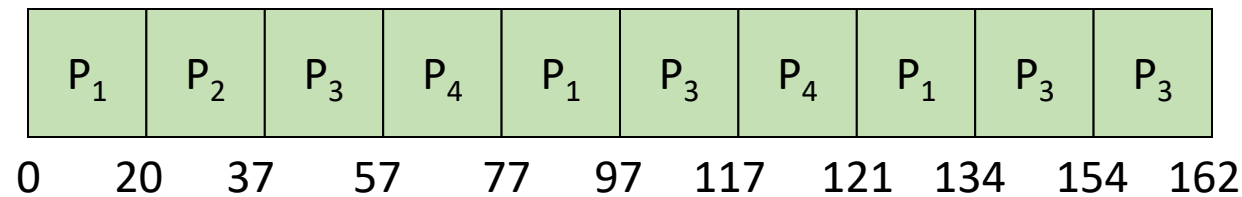
Round Robin Scheduling

- For RR scheduling we keep the ready queue as a FIFO queue of processes
- The CPU picks up the first process from the ready queue, set a timer to interrupt after 1 quantum
- If the process burst time is less than 1 quantum
 - then the process itself release the CPU voluntarily & proceed to next process
- If the process burst time is greater than 1 quantum
 - then that process is sent back to the ready queue and CPU scheduler selects the next process in the ready queue for scheduling

Round Robin Scheduling

<u>Process</u>	<u>Burst Time</u>
P ₁	53
P ₂	17
P ₃	68
P ₄	24

- Time quantum = 20 ms
- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better response

Priority Scheduling

- In priority scheduling a priority is associated with each process and the CPU is allocated to the process with the highest priority
- Equal priority processes are scheduled in FCFS order
- Priorities are generally some fixed range of numbers, such as 0, 1, 5, 10
- There is no general agreement on whether '0' is highest or lower priority
- Priority can be defined on the bases of following
 - Process time limit
 - Memory requirements
 - Number of open files
 - Ratio of average I/O burst to CPU burst

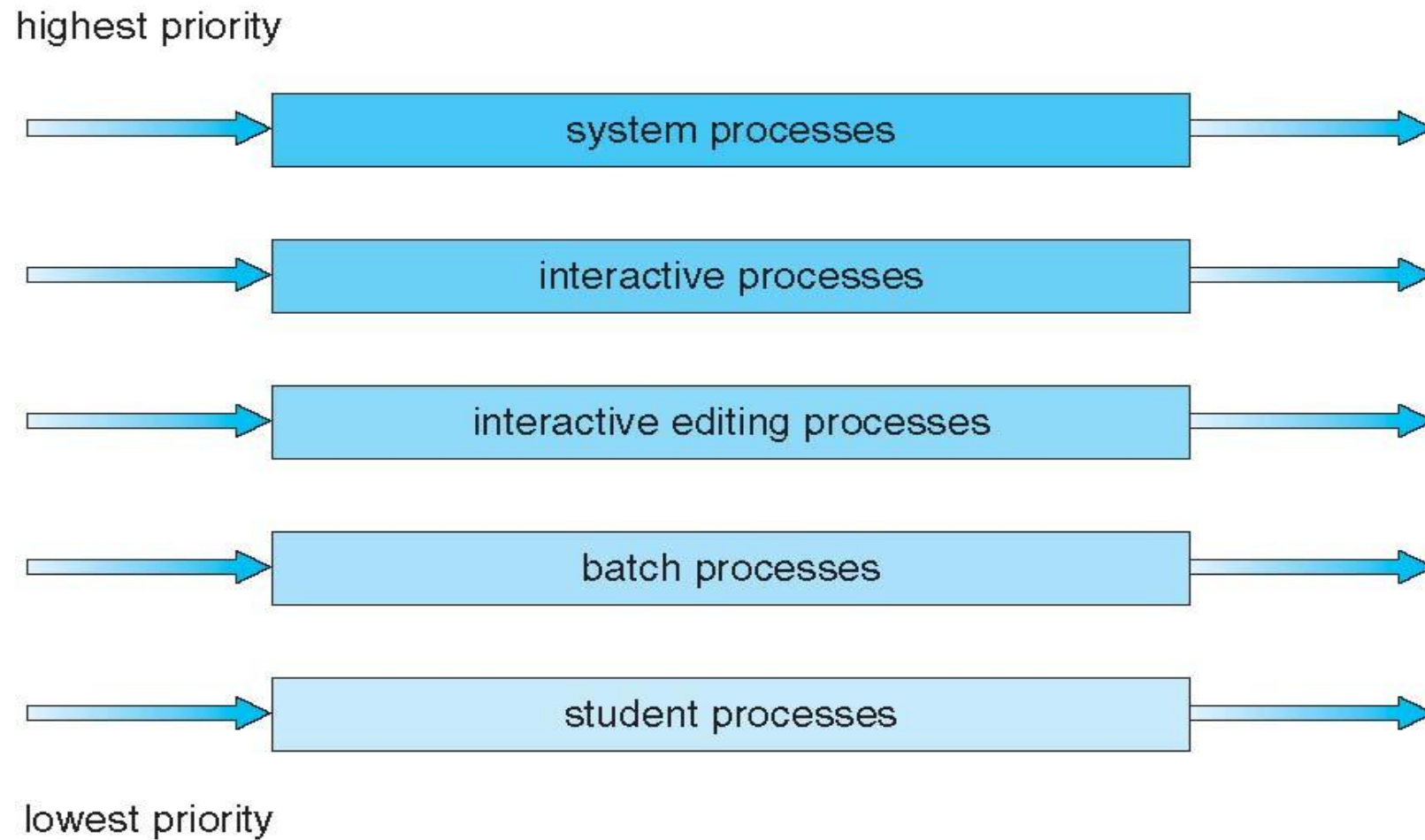
Priority Scheduling - Salient points

- Priority mechanism can be static or dynamic
 - Static priority mechanism is easy to implement and have relatively low overheads
 - Dynamic priority mechanism is more complex to implement and have greater overheads
- The priority scheduling may be preemptive or non-preemptive
 - The choice arise when a high priority process comes in the queue while a low priority process is executing
- A process that is ready to run but lacking the CPU can be considered blocked
- Solution to indefinite blocking is aging
- Aging is a technique of gradually increasing the priority of processes that waits for a long time
- OS allows only a limited priority classes
- In VAX / VMS priority range is 0 to 31
- In Unix priority range is -20 to 20

Multilevel Queue

- Ready queue is partitioned into separate queues:
 - foreground (interactive)
 - background (batch)
- Each queue has its own scheduling algorithm
 - foreground – RR
 - background – FCFS
- Scheduling must be done between the queues
 - Fixed priority scheduling; (i.e., serve all from foreground then from background)
 - Possibility of starvation
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR
 - 20% to background in FCFS

Multilevel Queue Scheduling



Multilevel Feedback Queue

- A process can move between the various queues
 - aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service

Multilevel Feedback Queue

Example

- Three queues:
 - Q_0 – time quantum 8 milliseconds
 - Q_1 – time quantum 16 milliseconds
 - Q_2 – FCFS
- Scheduling
 - A new job enters queue Q_0 which is served FCFS
 - When it gains CPU, job receives 8 milliseconds
 - If it does not finish in 8 milliseconds, job is moved to queue Q_1
 - At Q_1 job is again served FCFS and receives 16 additional milliseconds
 - If it still does not complete, it is preempted and moved to queue Q_2

Multilevel Feedback Queues

